



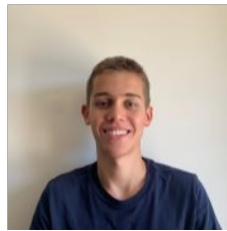
Compilador Fortran 77

Grupo 39

Universidade do Minho
Licenciatura em Engenharia Informática
Processamento de Linguagens – 2025/2026



Gonçalo Costa
A107381



Diogo Costa
A107328



Lourenço Martins
A106849

Conteúdo

1	Introdução	3
2	Opções de Implementação	3
2.1	Formato do código Fortran	3
2.2	Conversão para maiúsculas	3
2.3	Construções suportadas	3
3	Arquitectura do Compilador	4
4	Análise Léxica	4
5	Análise Sintática	5
5.1	Problema das linhas	5
5.2	Ambiguidade FOO(X) — array ou função?	5
5.3	Gramática (excerto)	5
6	Análise Semântica	6
6.1	Modelo de memória	6
6.2	Suporte a subprogramas	6
7	Geração de Código	6
7.1	Exemplo: programa Olá Mundo	6
7.2	Exemplo: ciclo DO	7
7.3	Convenção de chamada para funções	7
8	Optimização de Código	8
9	Testes	8
9.1	Testes automáticos	8
9.2	Validação na EWVM	9
10	Limitações	10
11	Conclusão	11

Lista de Figuras

1	Pipeline de compilação.	4
2	Resultado da execução dos testes automáticos.	9
3	Compilação do exemplo <code>factorial.f</code> no terminal.	9
4	Execução do <code>factorial.vm</code> na EWVM com entrada 5.	10
5	Execução do <code>conversor.vm</code> na EWVM com entrada 10.	10

1 Introdução

Este trabalho foi desenvolvido no âmbito da unidade curricular de Processamento de Linguagens e consiste na implementação de um compilador para um subconjunto do Fortran 77 (standard ANSI X3.9-1978).

O objectivo principal foi que o compilador fosse capaz de analisar código Fortran, validar a sua estrutura e gerar código para a máquina virtual EWVM disponibilizada para a unidade curricular.

O compilador foi implementado em Python, utilizando a biblioteca PLY (*Python Lex-Yacc*) tanto para a análise léxica como para a análise sintática, conforme pedido no enunciado.

2 Opções de Implementação

2.1 Formato do código Fortran

O Fortran 77 standard usa um formato de colunas fixas: a coluna 1 é reservada para comentários, as colunas 1 a 5 para *labels* numéricas, a coluna 6 para continuação de linha, e o código propriamente dito começa na coluna 7.

Optámos por não implementar este formato à letra, uma vez que é rígido e pouco prático de tratar. Em vez disso, suportamos um formato livre com as seguintes regras:

- linhas que comecem com `C`, `c` ou `*` são comentário (mantemos compatibilidade com o estilo clássico);
- o carácter `!` inicia um comentário até ao fim da linha;
- *labels* numéricas podem aparecer no início de qualquer linha, seguidas de espaço;
- o resto do código é lido em formato livre.

2.2 Conversão para maiúsculas

Fora de *strings*, o pré-processador converte todo o texto para maiúsculas. Assim, `program`, `PROGRAM` e `Program` são todos tratados da mesma forma, sem ser necessário lidar com capitalização no lexer ou no parser.

2.3 Construções suportadas

O compilador suporta as seguintes construções:

- declaração de variáveis `INTEGER`, `REAL` e `LOGICAL`, escalares e arrays;
- expressões aritméticas, relacionais e lógicas;
- atribuições;
- `IF ... THEN`, `ELSE IF`, `ELSE` e `ENDIF`;
- ciclos `DO` com *label* de fim e *step* opcional;
- `GOTO`, `CONTINUE`, `STOP` e `RETURN`;
- `PRINT *`, `...` e `READ *`, `...`;
- *builtins* `MOD` e `ABS`;
- definição e chamada de `FUNCTION` e `SUBROUTINE`.

3 Arquitectura do Compilador

O compilador está organizado em módulos separados, cada um responsável por uma fase do processo de compilação:

```
1 fortran77_compiler/  
2   compiler.py           programa principal  
3   f77c/  
4     preprocess.py       pre-processamento do código fonte  
5     lexer.py            analise lexica (ply.lex)  
6     parser.py           analise sintatica (ply.yacc)  
7     ast_nodes.py        nos da AST  
8     semantics.py        analise semantica  
9     codegen.py          geracao de código EWVM  
10    optimize.py         optimizacao peep-hole  
11    examples/           exemplos .f e respectivos .vm  
12    tests/              testes automaticos
```

O processo de compilação segue um *pipeline* com seis fases principais, ilustrado na Figura 1.

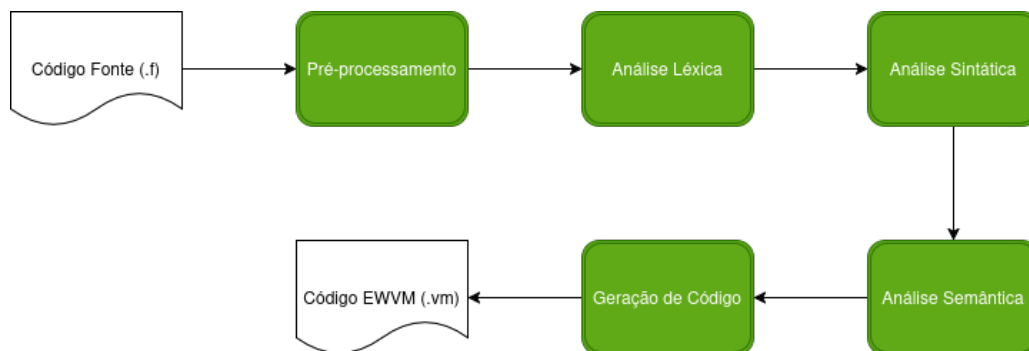


Figura 1: Pipeline de compilação.

As fases são:

1. **Pré-processamento:** remove comentários, converte para maiúsculas e separa as *labels* numéricas do resto da linha;
2. **Análise léxica:** identifica os *tokens* do programa (palavras-chave, identificadores, números, etc.);
3. **Análise sintática:** valida a gramática e constrói a AST (*Abstract Syntax Tree*);
4. **Análise semântica:** constrói a tabela de símbolos, verifica tipos e valida *labels*;
5. **Geração de código:** percorre a AST e produz instruções EWVM;
6. **Optimização:** aplica transformações peep-hole ao código gerado.

4 Análise Léxica

A análise léxica foi implementada em `f77c/lexer.py` usando `ply.lex`. O lexer recebe uma linha de texto já pré-processada e devolve uma sequência de *tokens*.

As palavras reservadas identificadas incluem PROGRAM, INTEGER, REAL, LOGICAL, IF, THEN, ELSE, DO, GOTO, PRINT, READ, FUNCTION, SUBROUTINE, entre outras.

Os operadores relacionais e lógicos do Fortran usam uma sintaxe com pontos (`.EQ.`, `.NE.`, `.AND.`, `.OR.`, `.NOT.`, etc.), o que obriga a definir regras próprias para os reconhecer. Por exemplo:

```
1 def t_AND(t):
2     r"\.AND\."
3     return t
4
5 def t_EQ(t):
6     r"\.EQ\."
7     return t
```

As *strings* do Fortran usam plicas simples, e uma plica dupla dentro da string (‘’) representa um apóstrofe. Esta lógica foi tratada tanto no pré-processador como na regra do token `STRING`.

5 Análise Sintática

5.1 Problema das linhas

O Fortran é uma linguagem orientada a linhas: a estrutura sintática depende de onde acabam as instruções. O PLY, por omissão, não dá essa informação ao parser.

A solução que encontrámos foi escrever um *wrapper* ao redor do lexer (`LineLexer` em `parser.py`) que injeta dois tipos de tokens artificiais:

- `LABEL`: injectado antes dos restantes tokens, quando uma linha começa com uma *label* numérica;
- `EOL`: injectado no fim de cada linha, permitindo que a gramática use este token como separador de instruções.

5.2 Ambiguidade `FOO(X)` — array ou função?

Em Fortran 77, a sintaxe para chamar uma função e para indexar um array é exactamente a mesma: `FOO(X)`. O parser não consegue distinguir os dois casos sozinho.

A solução foi que o parser produz sempre um nó `CallExpr`. Mais tarde, na análise semântica, esse nó é convertido num `ArrayRef` se o nome corresponder a um array declarado, ou mantido como chamada de função caso contrário.

5.3 Gramática (excerto)

A gramática completa está em `f77c/parser.py`. Apresenta-se aqui um excerto das regras principais:

```
1 program      : PROGRAM ID EOL decl_list stmt_list END subprogram_list
2
3 stmt         : opt_label simple_stmt EOL
4               | opt_label if_stmt
5
6 simple_stmt  : assignment | print_stmt | read_stmt
7               | goto_stmt  | do_stmt   | call_stmt
8               | continue_stmt | stop_stmt | return_stmt
9
10 if_stmt     : IF '(' expr ')' THEN EOL stmt_list if_tail
11 if_tail    : ENDIF EOL
12             | ELSE EOL stmt_list ENDIF EOL
13             | elseif_list opt_else ENDIF EOL
```

```

14
15 do_stmt      : DO INT_CONST ID '=' expr ',' expr
16              | DO INT_CONST ID '=' expr ',' expr ',' expr

```

A precedência dos operadores foi declarada explicitamente no parser (tabela `precedence`), do mais baixo para o mais alto: `.OR.`, `.AND.`, `.NOT.`, operadores relacionais, `+/` binários, `*/`, e por fim os operadores unários.

6 Análise Semântica

A análise semântica, implementada em `f77c/semantics.py`, percorre a AST e realiza as seguintes verificações:

- construção da tabela de símbolos para o programa principal e para cada subprograma;
- atribuição de *offsets* em memória a cada variável;
- verificação de tipos nas expressões e propagação desses tipos pela AST;
- validação de *labels*: os `GOTO` e os ciclos `DO` só podem referenciar *labels* que existam no mesmo bloco;
- verificações específicas do Fortran: a variável de controlo do `DO` não pode ser modificada dentro do ciclo, o *step* não pode ser zero, etc.

6.1 Modelo de memória

As variáveis do programa principal são guardadas na área global da EWVM (`PUSHG/STOREG`). No início do programa emite-se `PUSHN N`, onde `N` é o número de células necessárias (variáveis do utilizador mais temporários internos).

Nos subprogramas, as variáveis locais ficam no *stack frame* actual (`PUSHL/STOREL`), com *offsets* positivos. Os parâmetros ficam em *offsets* negativos:

- subroutines: parâmetros em `-1`, `-2`, ...
- functions: o slot `-1` é reservado para o valor de retorno, por isso os parâmetros começam em `-2`.

6.2 Suporte a subprogramas

Para suportar *forward references* (o programa principal chama uma função que é definida a seguir no ficheiro), a análise semântica é feita em duas passagens: primeiro registam-se as assinaturas de todos os subprogramas, depois analisam-se os corpos.

7 Geração de Código

A geração de código está implementada em `f77c/codegen.py`. O processo consiste em percorrer a AST e emitir instruções EWVM para uma lista de *strings*, que no fim são unidas com caracteres de nova linha.

7.1 Exemplo: programa Olá Mundo

O programa mais simples do enunciado:

```

1      PROGRAM HELLO
2      PRINT *, 'Ola, Mundo!'
3      END

```

gera o seguinte código EWVM:

```

1 START
2 PUSHS "Ola, Mundo!"
3 WRITES
4 WRITELN
5 STOP

```

7.2 Exemplo: ciclo DO

O ciclo DO do Fortran suporta *step* positivo ou negativo. Para que o teste de paragem funcione em ambos os sentidos, o código gerado avalia o sinal do *step* em *runtime* e escolhe o ramo adequado. Por exemplo, para o ciclo do factorial:

```

1      DO 10 I = 1, N
2          FAT = FAT * I
3      10 CONTINUE

```

é gerado código do género:

```

1 PUSHI 1
2 STOREG 1          ; I = 1 (start)
3 PUSHG 0
4 STOREG 3          ; end_temp = N
5 PUSHI 1
6 STOREG 4          ; step_temp = 1
7 JUMP dotestfatorial0
8 dobodyfatorial0:
9     ; corpo do ciclo
10    ...
11 dotestfatorial0:
12 PUSHG 4
13 PUSHI 0
14 SUPEQ
15 JZ donegfatorial0 ; step < 0?
16 PUSHG 1
17 PUSHG 3
18 INFEQ
19 JZ doendfatorial0 ; I > N? sair
20 JUMP dobodyfatorial0
21 donegfatorial0:   ; ramo step negativo
22    ...
23 doendfatorial0:

```

7.3 Convenção de chamada para funções

A convenção adoptada para chamadas a FUNCTION é a seguinte:

1. empilham-se os argumentos;
2. empilha-se um valor de retorno inicial (zero);
3. chama-se a função com CALL;

- dentro da função, o resultado é escrito com `STOREL -1`, que corresponde ao slot reservado logo abaixo dos parâmetros;
- após o retorno, guarda-se o resultado num temporário, limpam-se os argumentos com `POP`, e volta a colocar-se o resultado no topo da *stack*.

8 Optimização de Código

Foi implementado um optimizador peep-hole simples em `f77c/optimize.py`. Este optimizador analisa o código gerado em janelas pequenas (2 a 3 instruções) e elimina padrões redundantes.

As transformações implementadas são:

- Aritmética identidade:** elimina `PUSHI 0` seguido de `ADD` ou `SUB`, e `PUSHI 1` seguido de `MUL` ou `DIV`. Estes padrões surgem frequentemente no cálculo de índices de arrays quando o *lower bound* é 1;
- Saltos redundantes:** elimina `JUMP X` quando o *label* `X` está na linha imediatamente a seguir. Este caso é comum em `IF` sem `ELSE`;
- Código morto:** remove instruções que aparecem a seguir a `JUMP`, `STOP` ou `RETURN` sem que haja um *label* entre elas.

As transformações são aplicadas em rondas (até 3) até o código estabilizar. A optimização pode ser desligada com a *flag* `--no-opt`.

9 Testes

9.1 Testes automáticos

Os testes estão em `tests/test_compiler.py` e são corridos com:

```
1 python -m unittest -v
```

Foram implementados 28 testes organizados em 6 grupos:

- TestExamplesDoEnunciado:** verifica que os 5 exemplos do enunciado compilam sem erros;
- TestExpressoes:** testa aritmética, lógica e conversões de tipos;
- TestControloDeFluxo:** testa `IF`, `DO` e `GOTO`;
- TestSemanticErrors:** confirma que os erros semânticos são correctamente detectados;
- TestSubprogramas:** testa `FUNCTION` e `SUBROUTINE`;
- TestPreprocess:** testa comentários e *strings* mal formadas.

```
goncalo@goncalo-vivobook: ~/Universidade/trabalhop1
bash: /home/goncalo/.ghcup/env: No such file or directory
(base) goncalo@goncalo-vivobook:~/Universidade/trabalhop1$ python -m unittest tests/test_compiler.py -v
test_do_loop_basico (tests.test_compiler.TestControloDeFluxo.test_do_loop_basico) ... ok
test_goto_e_label (tests.test_compiler.TestControloDeFluxo.test_goto_e_label) ... ok
test_if_else_compila (tests.test_compiler.TestControloDeFluxo.test_if_else_compila) ... ok
test_if_elseif_else (tests.test_compiler.TestControloDeFluxo.test_if_elseif_else) ... ok
test_array_usa_check (tests.test_compiler.TestExemplosDoEnunciado.test_array_usa_check) ... ok
test_factorial_e_inteiro (tests.test_compiler.TestExemplosDoEnunciado.test_factorial_e_inteiro) ... ok
test_hello_imprime_a_string (tests.test_compiler.TestExemplosDoEnunciado.test_hello_imprime_a_string) ... ok
test_todos_os_exemplos_compilam (tests.test_compiler.TestExemplosDoEnunciado.test_todos_os_exemplos_compilam) ... ok
test_comparacao_real (tests.test_compiler.TestExpressoes.test_comparacao_real) ... ok
test_conversao_int_para_real (tests.test_compiler.TestExpressoes.test_conversao_int_para_real) ... ok
test_divisao_real (tests.test_compiler.TestExpressoes.test_divisao_real) ... ok
test_expr_aritmetica_simples (tests.test_compiler.TestExpressoes.test_expr_aritmetica_simples) ... ok
test_operadores_logicos (tests.test_compiler.TestExpressoes.test_operadores_logicos) ... ok
test_optimize_e_default (tests.test_compiler.TestOptimizer.test_optimize_e_default) ... ok
test_comentario_com_asterisco (tests.test_compiler.TestPreprocess.test_comentario_com_asterisco) ... ok
test_comentario_com_excl (tests.test_compiler.TestPreprocess.test_comentario_com_excl) ... ok
test_comentario_fortran_classico_C (tests.test_compiler.TestPreprocess.test_comentario_fortran_classico_C) ... ok
test_string_nao_terminada (tests.test_compiler.TestPreprocess.test_string_nao_terminada) ... ok
test_alterar_variavel_de_do (tests.test_compiler.TestSemanticErrors.test_alterar_variavel_de_do) ... ok
test_array_indice_errado (tests.test_compiler.TestSemanticErrors.test_array_indice_errado) ... ok
test_atribuir_real_a_logical (tests.test_compiler.TestSemanticErrors.test_atribuir_real_a_logical) ... ok
test_do_step_zero (tests.test_compiler.TestSemanticErrors.test_do_step_zero) ... ok
test_goto_sem_destino (tests.test_compiler.TestSemanticErrors.test_goto_sem_destino) ... ok
test_redeclaracao (tests.test_compiler.TestSemanticErrors.test_redeclaracao) ... ok
test_variavel_nao_declarada (tests.test_compiler.TestSemanticErrors.test_variavel_nao_declarada) ... ok
test_call_a_funcao_inexistente (tests.test_compiler.TestSubprogramas.test_call_a_funcao_inexistente) ... ok
test_funcao_inteira (tests.test_compiler.TestSubprogramas.test_funcao_inteira) ... ok
test_subroutine_call (tests.test_compiler.TestSubprogramas.test_subroutine_call) ... ok
.....
Ran 28 tests in 0.042s
OK
(base) goncalo@goncalo-vivobook:~/Universidade/trabalhop1$
```

Figura 2: Resultado da execução dos testes automáticos.

9.2 Validação na EWVM

Para além dos testes automáticos, cada exemplo foi validado manualmente na EWVM. O processo foi o seguinte:

1. compilar o ficheiro `.f` com o compilador:

```
1 python compiler.py examples/factorial.f
```
2. abrir a EWVM em <https://ewvm.ep1.di.uminho.pt/>;
3. carregar o ficheiro `.vm` gerado;
4. introduzir o valor de entrada quando pedido e verificar o resultado.

```
(base) goncalo@goncalo-vivobook:~/Universidade/trabalhop1$ python compiler.py examples/factorial.f
python compiler.py examples/prime.f
python compiler.py examples/conversor.f
[ok] gerado examples/factorial.vm
[ok] gerado examples/prime.vm
[ok] gerado examples/conversor.vm
```

Figura 3: Compilação do exemplo `factorial.f` no terminal.

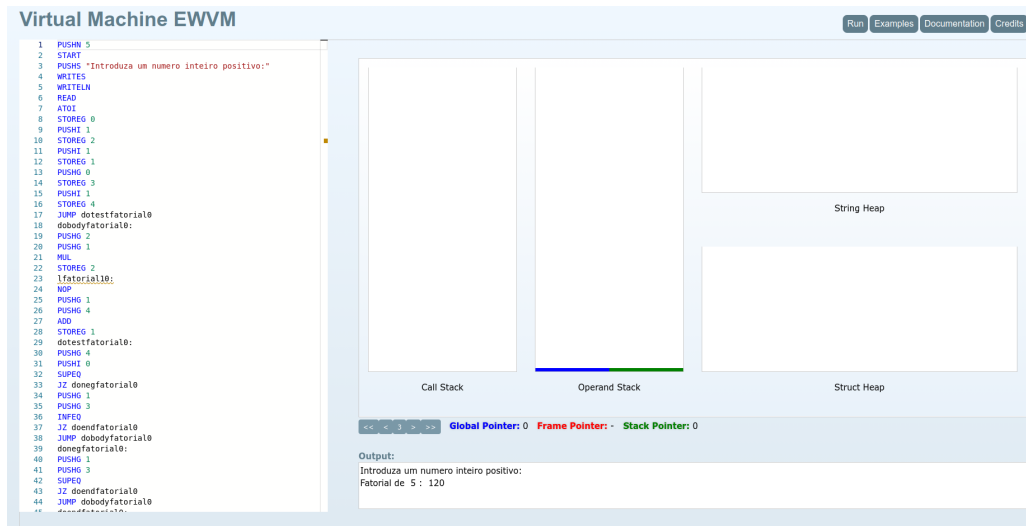


Figura 4: Execução do `factorial.vm` na EWVM com entrada 5.



Figura 5: Execução do `conversor.vm` na EWVM com entrada 10.

10 Limitações

O compilador não suporta algumas funcionalidades do Fortran 77 que ficaram fora do âmbito deste projecto:

- o tipo `CHARACTER` e variáveis de *string*;
- instruções `COMMON`, `DATA` e `EQUIVALENCE`;
- `IMPLICIT` (em Fortran 77 as variáveis que começam por I a N são `INTEGER` por omissão, mas não implementámos este comportamento);
- arrays passados como argumento de subprogramas;
- formatos no `PRINT/READ` — suportamos apenas o formato *list-directed* (*);
- o formato de colunas fixas estrito do standard.

11 Conclusão

O compilador implementado cobre as construções principais do Fortran 77 pedidas no enunciado, incluindo `FUNCTION` e `SUBROUTINE`, e foi validado com sucesso na máquina virtual EWVM para todos os exemplos fornecidos.

A organização em fases separadas facilitou o desenvolvimento e a identificação de problemas: quando algo falhava, era claro em que módulo era necessário intervir.

As maiores dificuldades encontradas foram a gestão dos tokens de fim de linha (necessários por o Fortran ser orientado a linhas), a ambiguidade entre acesso a array e chamada de função, e a convenção de chamada para funções com valor de retorno.